Exercise 2

Computer Vision, Fall 2025

Name: *Chiara Zara*

Instructions:

- Return the answer in PDF and Jupyter Notebook formats.
- Return latest on Sunday 21.9 at 23.50 via Moodle.

Ex 2.1 Edge detection and image filtering (3 points)

No description has been provided for this image

(a) Implement convolution function

- Implement convolution function to filter images with the user-defined kernel.
- Expect kernel sizes to be 3x3, 5x5, 7x7 or 9x9.
- In this exercise, we use only 1-channel grayscale images.
- Convolution function will be used in exercises 2.1b and 2.1c.

In [1]:
```
# Import Python packages needed in exercise 2.1
from matplotlib import pyplot as plt
import numpy as np
import cv2
```

In [2]:
```
def convolution(img, kernel):
    """
    Args:
       img: 1-channel grayscale image
       kernel: kernel to use in convolution.

    Returns:
        Convolved image
    """
    # make sure the image is 2D
    if img.ndim != 2:
        raise ValueError("Image must be 1-channel grayscale")

    k = kernel.shape[0]       # kernel size
    pad = k // 2              # padding around image

    # add padding with zeros
    img_pad = np.pad(img, ((pad, pad), (pad, pad)), mode='constant', constant_va

    # prepare empty output image
    out = np.zeros_like(img, dtype=np.float32)
```

```
    # slide kernel over every pixel
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            # select local region
            region = img_pad[i:i+k, j:j+k]
            # element-wise multiply and sum
            out[i, j] = np.sum(region * kernel)

    # clip values to 0-255 and convert to uint8
    out = np.clip(out, 0, 255)

    return out.astype(np.uint8)
```

(b) Implement create_box_filter function and filter image

- Implement function to create different size box filters
- Read image "images/norway.jpeg" and convert it to grayscale
- Use your convolution function to filter the image with box filters
- Try different box filter sizes. How does the box size affect the result?

Answer

I tried different box filter sizes to see how they change the image:

- The 3x3 filter gives a little blur but the image is still sharp.
- The 5x5, 7x7, 9x9 filters blur more, so some details disappear.
- The 15x15 filter makes the image very smooth and soft, almost like out-of-focus.

So, bigger box size means stronger blur and less detail.

In [3]:
```
def create_box_filter(size):
    """
    Args:
        size: Integer that defines size of the box filter.

    Returns:
        box filter with shape size x size.
    """
    box = np.ones((size, size), np.float32)
    box = box / (size * size)
    return box

#image in grayscale
img = cv2.imread("images/norway.jpg", cv2.IMREAD_GRAYSCALE)

# try different box sizes
sizes = [3, 5, 7, 9, 15]

plt.figure(figsize=(12,6))

for i, s in enumerate(sizes):
    #box filter
    box = create_box_filter(s)
    #convolution
    res = convolution(img, box)
```

```
    # show result
    plt.subplot(1, len(sizes), i+1)
    plt.title(f"Box {s}x{s}")
    plt.imshow(res, cmap='gray')
    plt.axis("off")

plt.show()
```

(c) Implement create_gaussian_filter function and filter image

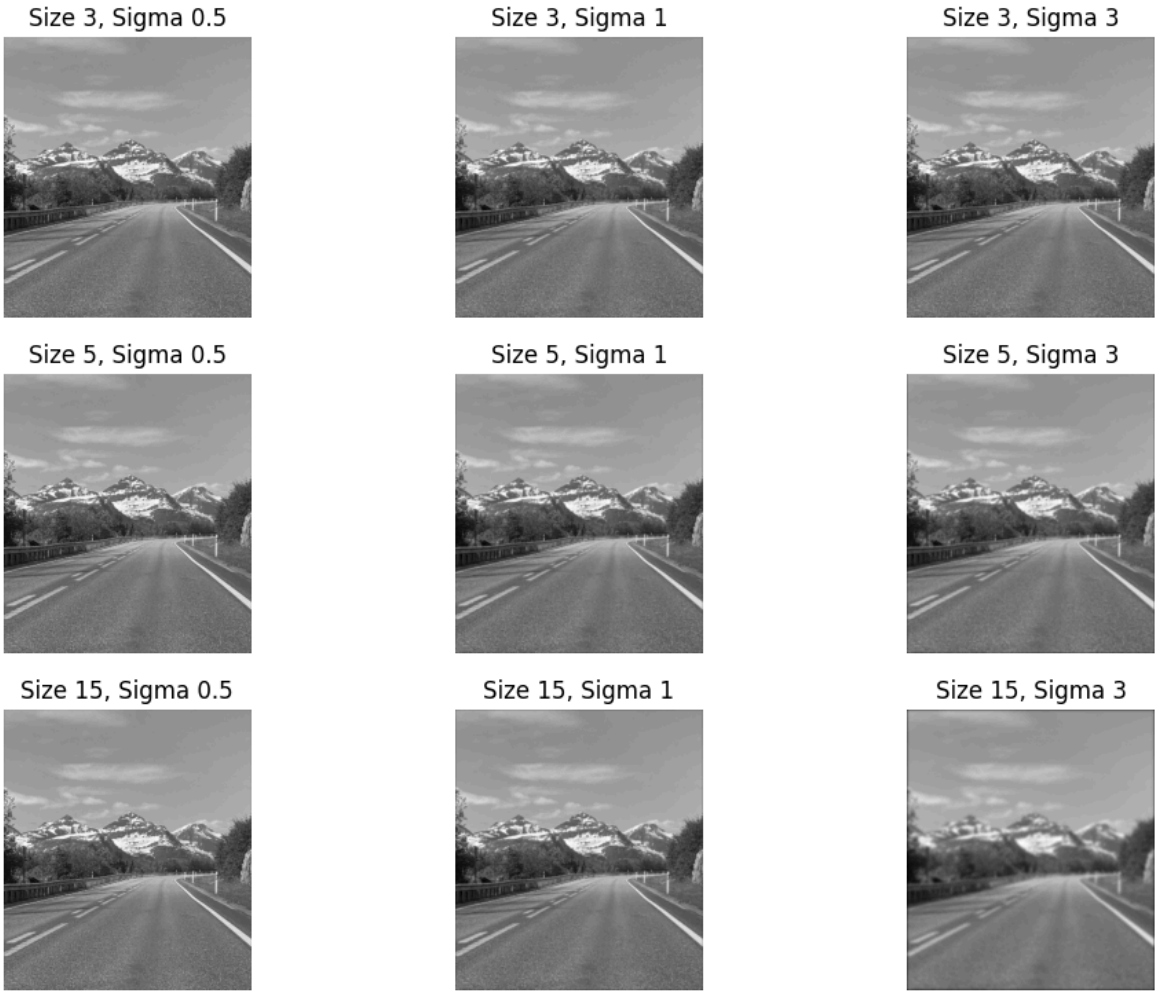
- Implement function to create a gaussian filters
- Read image images/norway.jpeg and convert it to grayscale
- Use your convolution function to filter the image with the different gaussian filters
- How does the change of sigma value and kernel size affect the results? Try with different values.

Answer

When sigma is small, the image is only a little blurred, even with a big kernel. With larger sigma, the blur becomes stronger and many details disappear. A bigger kernel does not make the blur stronger but makes it spread over a wider area.

In [27]:
```
def create_gaussian_filter(size, sigma):
    """
    Args:
        size: Integer that defines size of the gaussian filter.
        sigma: Standard deviation of Gaussian distribution

    Returns:
        gaussian filter with shape size x size and user defined sigma
    """
    #odd size
    if size % 2 == 0:
        raise ValueError("Size must be odd")

    #center of the kernel
    k = size // 2
    x, y = np.meshgrid(np.arange(-k, k+1), np.arange(-k, k+1))

    #gaussian formula
    g = np.exp(-(x**2 + y**2)/(2*sigma**2))

    # normalize so sum = 1
    g = g / np.sum(g)
    return g

img = cv2.imread("images/norway.jpg", cv2.IMREAD_GRAYSCALE)
```

```python
# define different kernel sizes and sigma values
sizes = [3, 5, 15]
sigmas = [0.5, 1, 3]

plt.figure(figsize=(12, 9))

count = 1
for s in sizes:
    for sigma in sigmas:
        # create gaussian kernel
        g = create_gaussian_filter(s, sigma)
        # apply convolution
        res = convolution(img, g)

        # show result
        plt.subplot(len(sizes), len(sigmas), count)
        plt.title(f"Size {s}, Sigma {sigma}")
        plt.imshow(res, cmap='gray')
        plt.axis("off")
        count += 1

plt.show()
```



(d) Edge detection

- Search edges from the original image and from all filtered images using OpenCV Canny edge detector. Visualize results.

- Which filter provides the best result? Why? Consider the best result to be one that could be used by the autonomous vehicle to stay in its lane (clear line markings and minimal amount of image noise).

Hint:

- OpenCV Canny Edge Detection

Answer

The Gaussian filter gives the best result. It works better than the box filter because it reduces noise while keeping edges sharp. As we can see from the immages, the box filter blurs the image too much, making lane lines unclear. For an autonomous car, it is important to see the lanes clearly and avoid mistakes, so Gaussian is preferred because it keeps the lines continuous and reduces unnecessary noise.

In [19]:
```
img = cv2.imread("images/norway.jpg", cv2.IMREAD_GRAYSCALE)
#Box filter
box_filtered = cv2.blur(img, (9,9))
#Gaussian filter
gaussian_filtered = cv2.GaussianBlur(img, (9,9), 2)
#Canny edge detector
e_box = cv2.Canny(box_filtered, 50, 150)
e_gaus = cv2.Canny(gaussian_filtered, 50, 150)
#Show results
plt.figure(figsize=(8,6))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title('Main')
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(e_box, cmap='gray')
plt.title('Box Filter')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(e_gaus, cmap='gray')
plt.title('Gaussian Filter')
plt.axis('off')

plt.show()
```

Ex 2.2 Detect and match features (3 p)

(a) Detect and visualize SIFT features

- Read images "images/GOPR1515_03850" and "images/GOPR1515_03852" and convert them to grayscale
- Run OpenCV SIFT feature detection
- Visualize found SIFT keypoints from both images

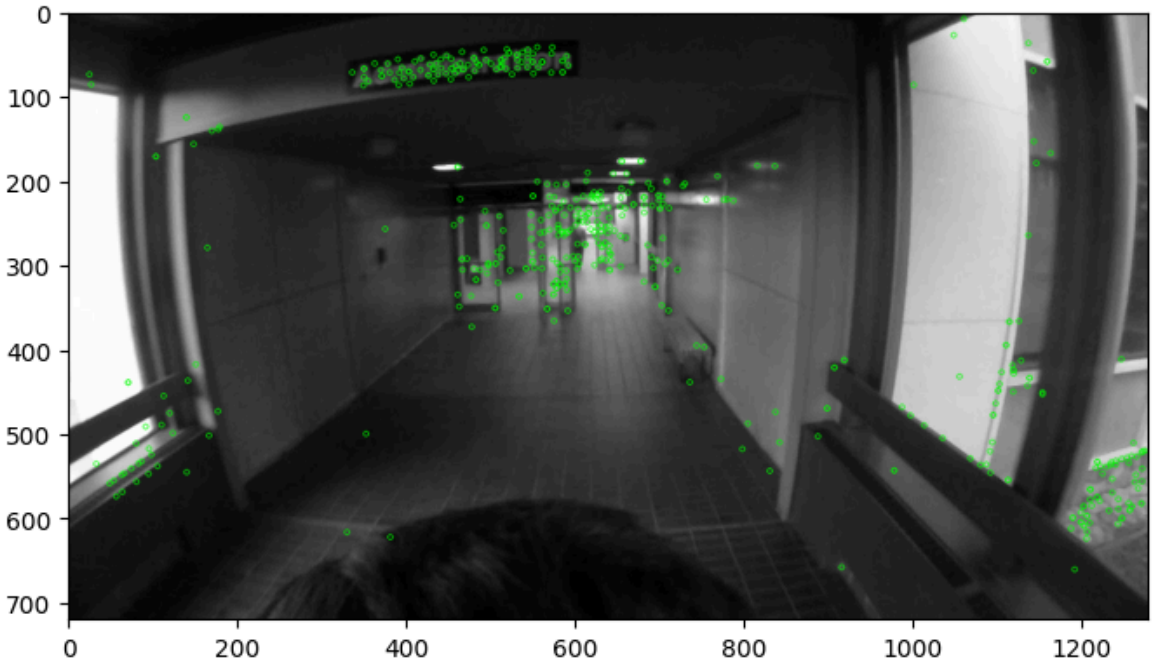
Hints:

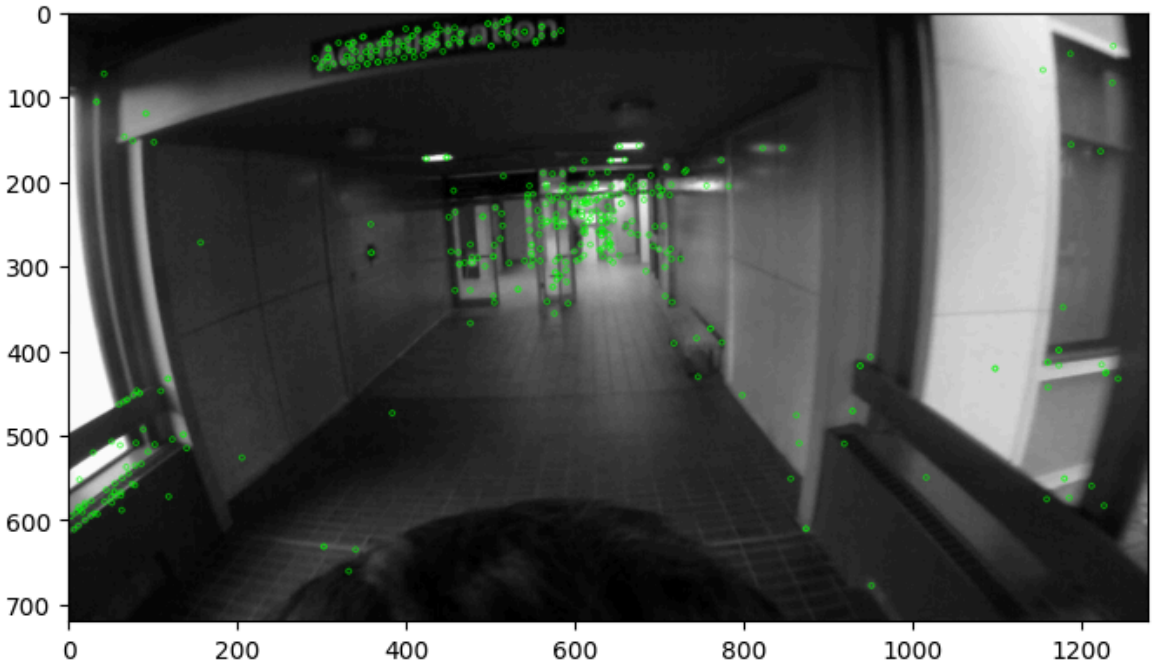
- We provide you a function visualize_features which can be used to visualize your results.
- Depending on your OpenCV version, you may need to install OpenCV contrib package: Link to opencv-contrib-python
- OpenCV material that may be useful:
  - OpenCV cv::KeyPoint Class Reference
  - OpenCV cv::Feature2D Class Reference
  - Introduction to SIFT (Scale-Invariant Feature Transform in OpenCV)

In [29]:
```
def visualize_features(image, kp):
    """
    Visualize extracted features in the image

    Arguments:
       image: a grayscale image
       kp: list of the extracted keypoints

    Returns:
    """
    display = cv2.drawKeypoints(image, kp, None, color=(0,255,0), flags=0)
    plt.figure(figsize=(8, 6), dpi=100)
    plt.imshow(display)
```

In [30]:
```
#read immages and convert them to greyscale
img1 = cv2.imread("images/GOPR1515_03850.jpg")
img2 = cv2.imread("images/GOPR1515_03852.jpg")
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
```

```python
#Run OpenCV SIFT feature detection
sift = cv2.SIFT_create()

#keypoints and descriptors
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

#Visualize found SIFT keypoints from both images
visualize_features(gray1, kp1)
visualize_features(gray2, kp2)
```





(b) Feature matching

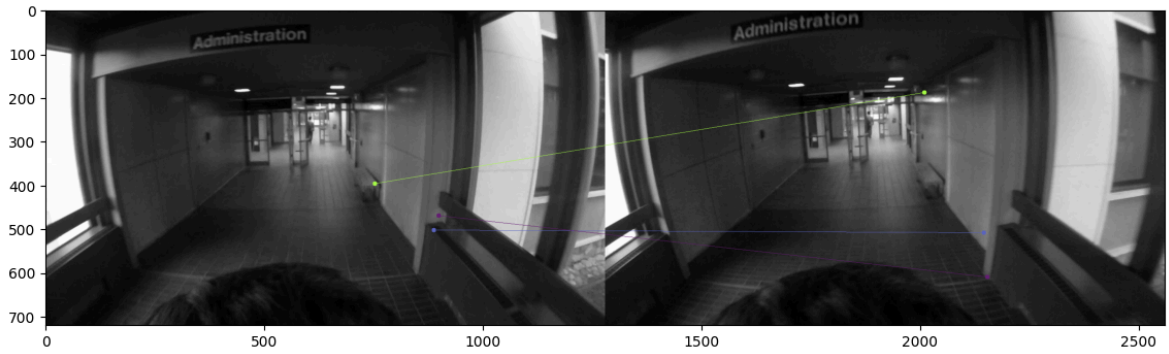
- Implement feature matching algorithm (you can use OpenCV matching algorithm only to verify your results).
- How good is the result? How could you improve it?

Hint:

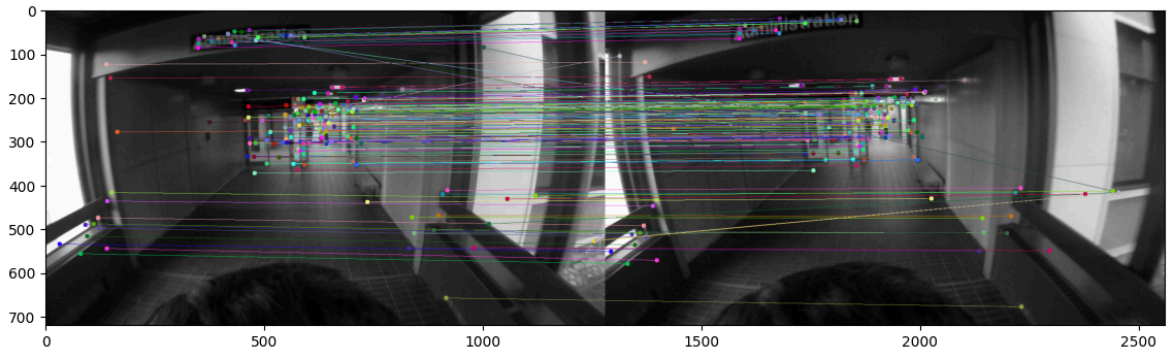
- We provide you a function visualize_matches which can be used to visualize your results.
- OpenCV Feature Matching tutorial. Do not use ready made matching algorithm in this exercise!

In [33]:
```
import random
def visualize_matches(image01, kp01, image02, kp02, matches):
    """
    Visualize corresponding matches in two images

    Arguments:
        image1: the first image in a matched image pair
        kp1: list of the keypoints in the first image
        image2: the second image in a matched image pair
        kp2: list of the keypoints in the second image
        matches: list of keypoint index pairs of matched features. Example forma

    Returns:
    """
    # Get width of the first image
    _ , width = image01.shape

    # Concatenate images
    img = cv2.hconcat([image01, image02])

    # Convert image from gray to RGB
    img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)

    # Add matches to image
    for match in matches:

        # Get random color
        r = random.randint(0,255)
        g = random.randint(0,255)
        b = random.randint(0,255)
        color = (r,g,b)

        # Get coordinates
        point_img01 = kp01[match[0]].pt
        point_img02 = kp02[match[1]].pt

        # Get x and y values from coordinates
        x_img01 = int(point_img01[0])
        y_img01 = int(point_img01[1])
        x_img02 = int(point_img02[0]) + width
        y_img02 = int(point_img02[1])

        # Draw small circle to matched keypoints
        cv2.circle(img, (x_img01, y_img01), radius=5, color=color, thickness=-1)
        cv2.circle(img, (x_img02, y_img02), radius=5, color=color, thickness=-1)

        # Draw line between matches
        cv2.line(img, (x_img01, y_img01), (x_img02, y_img02), color, thickness=1

    # Show image with matches
    plt.figure(figsize=(14, 14), dpi=100)
```

```
    plt.imshow(img)

# Dummy matches to test visualize_matches function.
# You can try it after you have read images and extracted keypoints in exercise
matches = [(382, 422), (384, 424), (368, 407)]
visualize_matches(gray1, kp1, gray2, kp2, matches)
```



In [34]:
```
#Load images and grayscale
img1 = cv2.imread("images/GOPR1515_03850.jpg")
img2 = cv2.imread("images/GOPR1515_03852.jpg")
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

#SIFT keypoints
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

#matching with L2 distance + ratio test
matches = []
for i, d1 in enumerate(des1):
    best = -1
    best_dist = 1e10
    second = 1e10
    for j, d2 in enumerate(des2):
        dist = np.linalg.norm(d1-d2)
        if dist < best_dist:
            second = best_dist
            best_dist = dist
            best = j
        elif dist < second:
            second = dist
    if best_dist < 0.75 * second:
        matches.append((i, best))

#Visualize matches
visualize_matches(gray1, kp1, gray2, kp2, matches)
```



Ex 2.3: Fourier Transform (2 points)

Helpful resources:

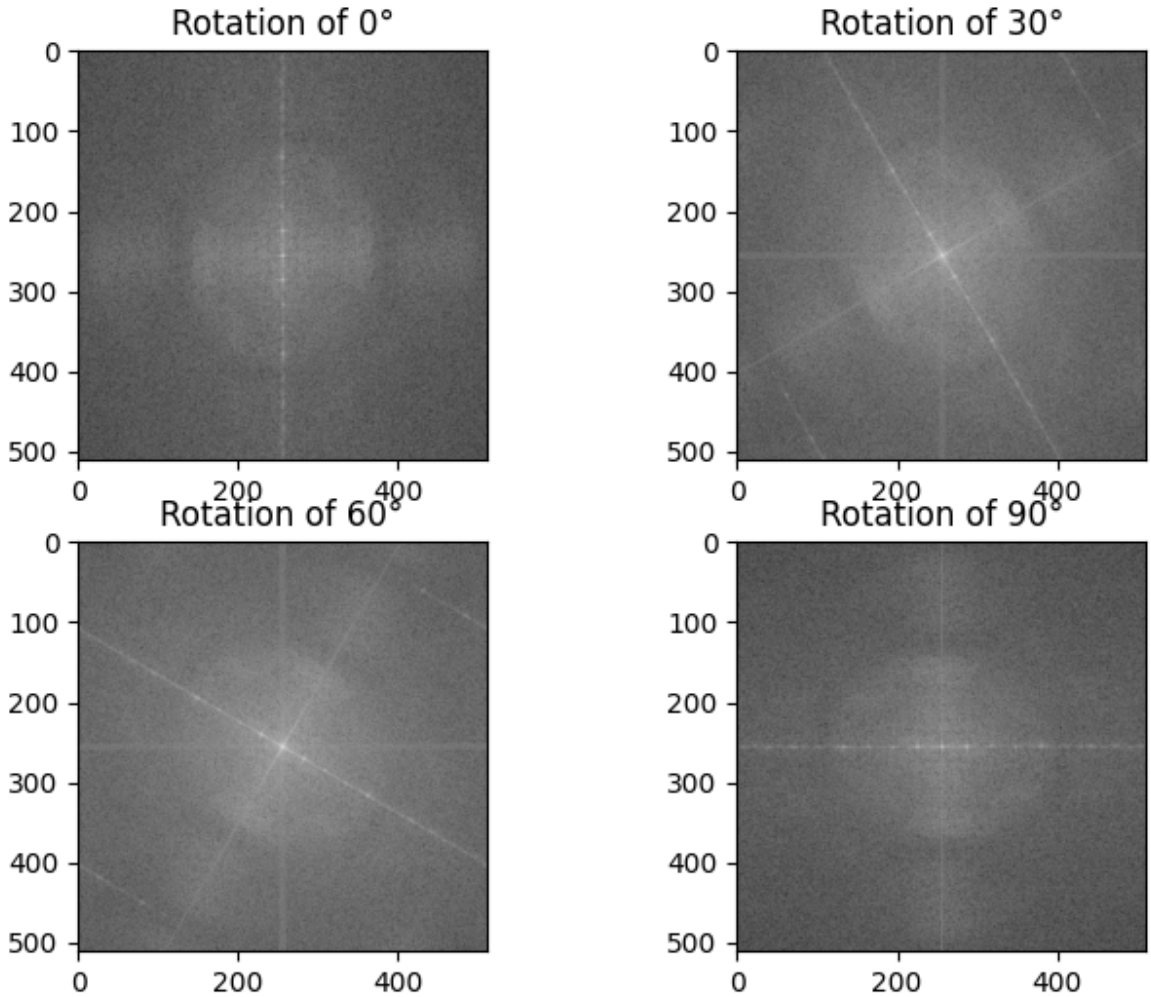
- You may utilize either OpenCV or NumPy for FFT and image rotation operations. You can use ready-made functions, don't have to implement FFT from scratch.
- Refer to this link for a detailed OpenCV Fourier Transform tutorial: OpenCV Fourier Transform Tutorial

Task (a): Evaluating Image Orientation in Signal Space

1. Load the image 'images/son3.png'.
2. Create three copies of the image: one in the original orientation, one rotated by 30 degrees, another by 60 degrees, and the last one by 90 degrees.
3. Perform the Fast Fourier Transform (FFT) followed by an FFT shift on all four images. Then visualize their magnitude spectrums.
4. Observe and describe the orientation of the text in the magnitude images.
5. Explain what causes the appearance of horizontal and vertical lines in the FFT magnitude spectrum. What is the x-axis and y-axis in the frequency domain?

In [18]:
```
img = cv2.imread('images/son3.png', cv2.IMREAD_GRAYSCALE)
#function to rotate image around center
def rotate_img(image, angle):
    h, w = image.shape
    M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1)
    rotated = cv2.warpAffine(image, M, (w, h))
    return rotated

#create rotated versions
img0 = img
img30 = rotate_img(img, 30)
img60 = rotate_img(img, 60)
img90 = rotate_img(img, 90)

def fft_magnitude(image):
    f = np.fft.fft2(image)
    fshift = np.fft.fftshift(f)
    magnitude = 20*np.log(np.abs(fshift) + 1)  # +1 to avoid log(0)
    return magnitude

mag0 = fft_magnitude(img0)
mag30 = fft_magnitude(img30)
```

```
mag60 = fft_magnitude(img60)
mag90 = fft_magnitude(img90)

plt.figure(figsize=(8,6))

plt.subplot(2,2,1)
plt.imshow(mag0, cmap='gray')
plt.title('Rotation of 0°')

plt.subplot(2,2,2)
plt.imshow(mag30, cmap='gray')
plt.title('Rotation of 30°')

plt.subplot(2,2,3)
plt.imshow(mag60, cmap='gray')
plt.title('Rotation of 60°')

plt.subplot(2,2,4)
plt.imshow(mag90, cmap='gray')
plt.title('Rotation of 90°')

plt.show()
```



Answer:

4. The text orientation in the Fourier magnitude images changes in the same way as in the original images. When we rotate the image by 30°, 60° or 90°, the frequency

spectrum also rotates by the same angle and so the lines and patterns in the spectrum follow the rotation of the text.

5. The horizontal and vertical lines in the spectrum come from repeating patterns of the text. Text has many parallel edges, so these produce strong frequencies that appear like bright lines. In the frequency domain, the x-axis shows the horizontal frequencies and the y-axis shows the vertical frequencies.The center of the spectrum is the low frequency and so average brightness, and moving away from the center means higher frequencies and so details.

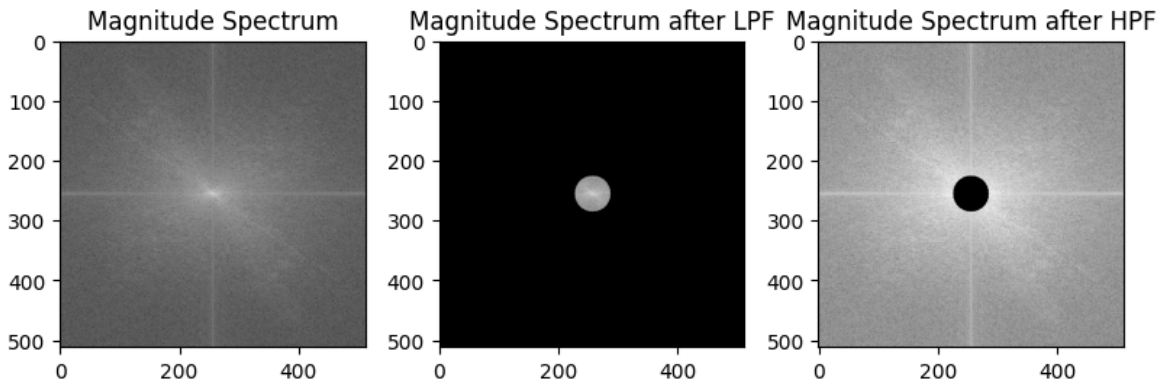
Task (b) High Pass Filter and Low Pass Filter

1. Open the image 'images/lenna.png' and convert it into grayscale.
2. Conduct a Fast Fourier Transform (FFT) on this image.
3. Next, create two copies of the Fourier magnitude spectrum. For one copy, apply a low pass filter (LPF) and for the other, apply a high pass filter (HPF). In both cases, utilize a circular mask with a 30-pixel radius for the filtering process. You can use the function create_circular_mask() below to create the mask for the filters.

In [5]:
```
def create_circular_mask(img, radius):
    # Get image size
    h, w = img.shape

    # Get center coordinate
    center = (int(w/2), int(h/2))

    # Make open multi-dimensional “meshgrid”
    Y, X = np.ogrid[:h, :w]

    # Calculate distance from the image center to every pixel
    dist_from_center = np.sqrt((X - center[0])**2 + (Y-center[1])**2)

    # Threshold the result with the user defined radius
    mask = dist_from_center <= radius

    # Convert mask from boolean array to integer array
    return mask.astype(int)
```

In [16]:
```
#load image and convert to gray
img = cv2.imread('images/lenna.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

#FFT
f = np.fft.fft2(gray)
fshift = np.fft.fftshift(f)
mag = np.log(1 + np.abs(fshift))
#masks
mask_lpf = create_circular_mask(gray, 30)
mask_hpf = 1 - mask_lpf
#apply masks to spectrum
mag_lpf = mag * mask_lpf
mag_hpf = mag * mask_hpf

#show magnitude spectra
plt.figure(figsize=(8,6))
```

```
plt.subplot(1,3,1)
plt.imshow(mag, cmap='gray')
plt.title('Magnitude Spectrum')

plt.subplot(1,3,2)
plt.imshow(mag_lpf, cmap='gray')
plt.title('Magnitude Spectrum after LPF')

plt.subplot(1,3,3)
plt.imshow(mag_hpf, cmap='gray')
plt.title('Magnitude Spectrum after HPF')

plt.tight_layout()
plt.show()
```



4. Below are images of what the magnitude spectrum should look like before and after applying the the filters. Briefly explain what the filters are actually doing to the image. How do you expect it to change the image after the image is converted back from the frequency domain?

comparison image

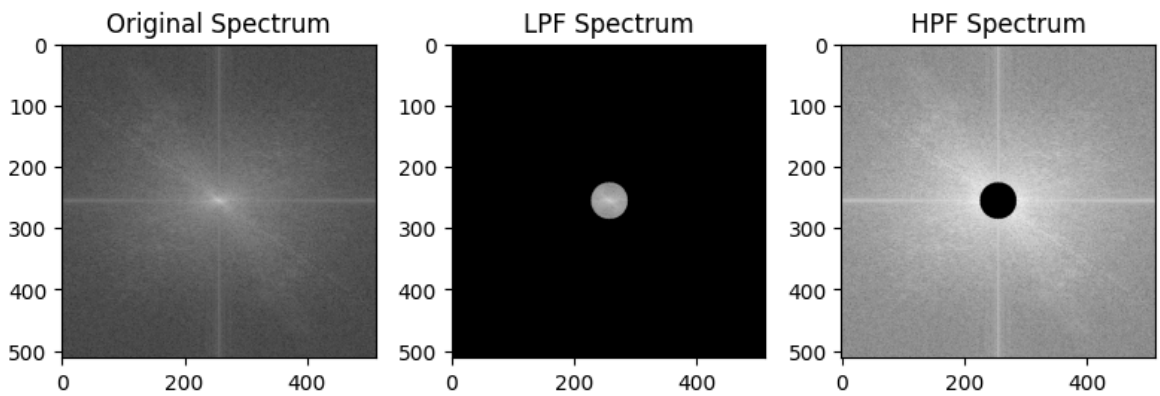
Answer:

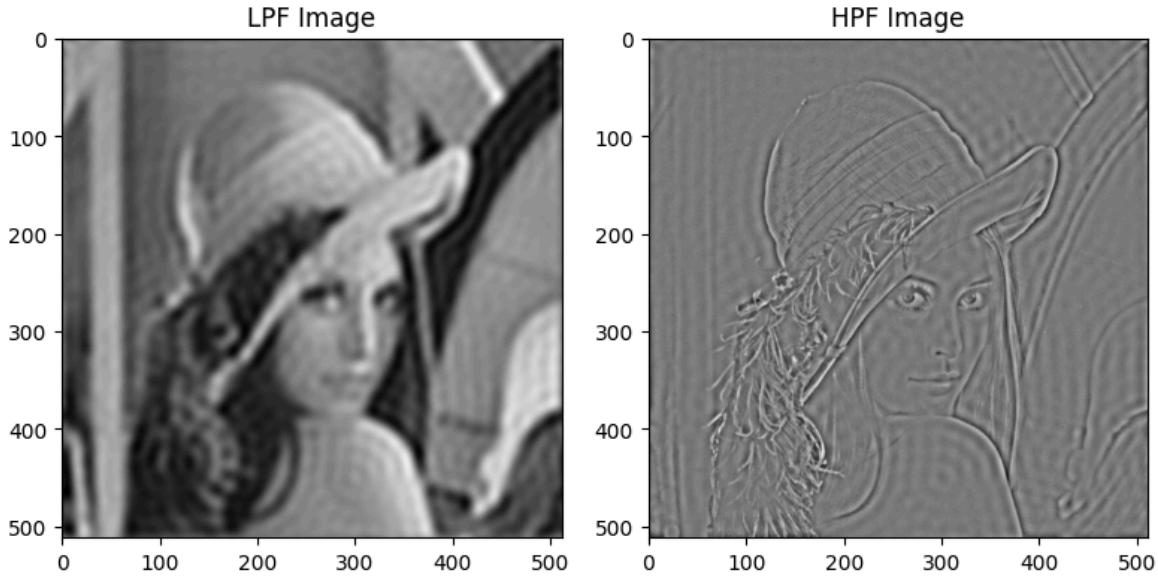
4. In the magnitude spectrum after LPF we can see a bright circle in the center and the rest is dark. This because the LPF (Low Pass Filter) lets only the low frequencies pass and blocks the high ones, making the image blurry because edges and small details are removed, keeping only smooth parts. The HPF (High Pass Filter) does the opposite: it blocks low frequencies and lets high frequencies pass. In the spectrum, the center is dark and the outer parts are bright. After applying it, the image looks sharper and edges are more visible, showing more details and textures.

5. Perform an inverse Fourier transform on the spectra that have been filtered.
6. Finally, observe and discuss the impact that the LPF and HPF have had on the image based on the answer you gave in part 4. of the assignment. What would be a possible application for using the High Pass Filter or the Low Pass Filter?

In [17]:
```
img = cv2.imread('images/lenna.png', cv2.IMREAD_GRAYSCALE)

#FFT
```

```
f = np.fft.fft2(img)
f_shift = np.fft.fftshift(f)
mag = np.log(1 + np.abs(f_shift))

#masks
radius = 30
mask = create_circular_mask(img, radius)

# LPF and HPF
f_lpf = f_shift * mask
f_hpf = f_shift * (1 - mask)

#magnitude spectra after filtering
mag_lpf = np.log(1 + np.abs(f_lpf))
mag_hpf = np.log(1 + np.abs(f_hpf))

#inverse FFT to get images back
img_lpf = np.fft.ifft2(np.fft.ifftshift(f_lpf)).real
img_hpf = np.fft.ifft2(np.fft.ifftshift(f_hpf)).real

#magnitude spectra
plt.figure(figsize=(8,6))
plt.subplot(131), plt.imshow(mag, cmap='gray'), plt.title('Original Spectrum')
plt.subplot(132), plt.imshow(mag_lpf, cmap='gray'), plt.title('LPF Spectrum')
plt.subplot(133), plt.imshow(mag_hpf, cmap='gray'), plt.title('HPF Spectrum')
plt.tight_layout()
plt.show()

#images after inverse FFT
plt.figure(figsize=(8,6))
plt.subplot(121), plt.imshow(img_lpf, cmap='gray'), plt.title('LPF Image')
plt.subplot(122), plt.imshow(img_hpf, cmap='gray'), plt.title('HPF Image')
plt.tight_layout()
plt.show()
```





Answer:

6. As said in answer 4, the LPF makes the image blurry because it removes high frequencies, which are edges and small details. Only smooth parts stay, so the image looks less detailed. The HPF does the opposite: it removes low frequencies and keeps high ones. This makes edges and textures more visible, so the image looks sharper and more detailed. HPF can be used for edge detection or image sharpening, since it emphasizes edges and small details that are important for tasks like feature detection or computer vision algorithms. LPF can be used to make images smoother or remove some noise.